

QR code based personal information sharing app

1. Introduction

As we live in a fast paced digitized world, sharing information is highly automatized. However, a specific data sharing domain remains underexplored, which is sharing efficiently and securely personal sensitive information and company business sensitive information. This report and project proposed a solution in this direction.

2. The data format

The data being transferred is structured data, similar to the information encoded by a vCard at its base, and is compatible with a form format. The data is organized as key – value pairs, using the json format.

The system will support sharing encrypted personal information using asymmetric keys. The shared information can be contact information similar to a vCard, or other additional sensitive personal information, as well as company information if the user is a business, and also larger files, like personal or business documents.

Example of a vCard style shared business information:

```
{  
  "company": "Acme Solutions",  
  "owner": "John Smith",  
  "phone": "+1-555-123-4567",  
  "email": "john@acmesolutions.com",  
  "website": "https://acmesolutions.com",  
  "address": "123 Main Street, New York, NY",  
}
```

Example of larger files:

- digital scans of personal or business documents
- contracts, NDAs
- financial information, etc.

When the shared information is just a few KB in size, it can be embedded directly into an QR code, encrypted, and shared with the intended receiver.

If the data is too large to fit into a QR code, or if a secure peer to peer connection is preferred instead for the data transfer, the data is being transferred over a secure peer to peer channel, also encrypted.

2. System architecture and design

The proposed architecture of the information sharing system is comprised a sender entity application, and a receiver entity application, in a peer to peer communication model, where the sender and the receiver can transfer information directly, with no intermediary entity, in one of two modes possible: direct data exchange through secured QR codes, or data exchange through secured peer to peer connection, where QR codes are used only for setting up the secure connection.

By exchanging information (data and encryption keys) embedded in QR codes which are generated on the spot, the sender app can share sensitive information which it stores locally on the device directly with the receiver app, by simply scanning the QR code through the receiver app.

For both the data transfer modes, the data storage is decentralized, being stored solely on the sender's device, locally, secured and encrypted. The data is managed by the sender app. The receiver app can get the information needed from the sender directly, after negotiating what data the sender allows to share. The data is either embedded directly in the sender's QR code, if it is low in size, or, if larger in size, by transferring it over a peer to peer direct connection established between sender and receiver, where the secured connection is being setup through the QR codes.

In the following sections, the 2 modes of sharing sensitive information using a peer to peer architecture are presented in larger details.

2.1. Direct data exchange through secured QR codes (vCard style)

In this approach, the client app creates a QR code which embeds all the information he wants to share, encrypted. A low secure mode is also permitted where the information in the QR code can be unencrypted, if the client wants this.

2.1.1. The data transfer protocol

The data transfer protocol is comprised of 3 or 4 steps. Either the sender or the receiver can initiate the transfer protocol.

Step 0 (optional): Sender offers to share arbitrary data with a receiver

If data transfer is initiated by the sender, i.e. the sender wants to share specific information with a receiver, undemanded by the receiver, the sender can create the data package (e.g.

json based form), and will create a QR code containing the meta data of the form package, i.e. what it includes, which he can offer a receiver to scan. Besides the form metadata, the QR code will also contain information about who the sender claims to be.

Step 1: The receiver wants to receive specific data from a sender

The receiver selects in the app the information it needs, and encodes this as metadata in a QR code, alongside with information about who the receiver is. When generating the QR code, a new asymmetric key pair is generated transparently in the app, and the public key is appended automatically to the QR code.

The sender will use the receiver's public key to encrypt the data inside his QR code, such that only the receiver who owns the matching private key can decrypt the shared data. The receiver cannot see the private key either, as it is managed internally by the receiver app, and it is destroyed automatically and immediately after the receiver gets the information asked through the sender's QR code which embeds the encrypted data.

Step 2: The sender scans the receiver's QR code to review the requested data, and generates the QR code which embeds the requested data, encrypted

The sender's app scans the receiver's QR code using the device's camera, which decodes it and displays to the user the requested data, the receiver's identity information, and also saves locally, temporarily the receiver's public key.

The user will check who the receiver claims to be, will review the requested data (some requested data can be marked by receiver as optional), having to ability to select which data he accepts to share.

If the sender user agrees to share data with the receiver, he selects the data he wants to share, or confirms he wants to share everything asked, and proceeds to generate the QR code which embeds all the data, encrypted using the receiver's shared public key.

Step 3: The receiver scans the sender's QR code to received the data

The receiver scans the sender's QR code, using the receiver app, which will decode the data contained by the QR code, will decrypt the data using the matching private key, will save the transferred data internally and will automatically delete the key pair, immediately after the transferred data was decrypted and saved.

2.1.2. Managing multiple transfers at the same time

Because the protocol is asynchronous and is based on scanning multiple QR codes, to support the management of the transfer "sessions" on the sender's app and on the receiver's app, a "session" like transfer key will also be embedded in the QR code with the

STEP 0 or STEP 1, and will be reused with all the following QR codes, by both the sender app and the receiver app, such that both ends can keep track of different transfers initiated.

2.2. Data transfer through secured peer 2 peer connection (dynamic vCard style).

This data transfer proposed method can be used to avoid embedding the data directly into the QR code itself, and also is needed when wanting to transfer more data that could be contained in a single QR code (usually QR codes can encode a few KB), for example useful for transferring digital copies of documents, by establishing a secured peer to peer connection between the sender and the receiver apps.

Step 0 and Step 1 remain the same. The only changes are in the Step 2, i.e. the sender's QR code containing data, and Step 3, i.e. the receiving of the data step by the receiver.

2.2.1. The data transfer protocol

Step 0 (optional): Sender offers to share arbitrary data with a receiver

This step works the same as in the case of the first method, i.e. direct data embedding in the QR.

Step 1: The receiver wants to receive specific data from a sender

This step is also identical as in the case of the first transfer method discussed in section 2.1.1.

Step 2: The sender scans the receiver's QR code to review the requested data, prepares the requested data, encrypted, and generates the QR code containing the location and how to connect securely to the sender's device to retrieve the data

To establish the secured connection, the sender's QR code will embed this time the URI of the sender temporary hosted data transfer service, which is hosted locally on the sender's device through the sender app. At that location, the sender's app placed the requested data package, encrypted with the receiver's public key.

Besides the URI, the parameters needed to secure the connection are also embedded in the QR code. All this information encoded in the sender's QR code is also encrypted with the receiver's public key, as before.

Step 3: The receiver scans the sender's QR code to get the connect details, connect to the sender's app, and receives the data

The receiver scans the sender's QR code through the receiver app. The app automatically decodes and decrypts using its matching private key the data embedded in the QR code. It

extracts the URI and the secure connection parameters, and connects to the sender's device in a peer to peer connection using those connection details. After downloading the package, the receiver decrypts it using its private key, saves the data internally, then the key pair is automatically deleted by the receiver app.

An authentication step can be used to enforce secure authentication before being able to download the package, where the receiver proves that he is the intended entity to download that file, by digitally signing a challenge provided at the URI, using the same private key as before. If verified correctly by sender using the receiver's public key, the download of the package can then be permitted.

3. Security design

Being a system designed specifically with the purpose of easily sharing sensitive personal and business information, over QR codes, the security posture of the system needs to be considered from the design phase, following the recommended practices of a secure-by-design architecture.

The security concerns span all over the system components, over the communication protocols used, encryption methods, data privacy and data protection and many other aspects. This section tackles the obvious security concerns of the system and will be extended to cover other aspects as well in the following reports.

3.1. Data privacy and protection

Being a system which deals mainly with sensitive information of physical persons and companies, stored locally on the device, embedded in the QR code, and transferred network connections, ensuring the protection and privacy of the data is crucial to the system.

3.1.1. Encryption at rest

Data is stored locally on the device of the data owner, encrypted. The encryption key is managed by the app solely, and is protected through a PIN, password, or biometric information.

3.1.2. MFA authentication support

The authentication will support user and password login, as well biometric authentication, and 3rd party authentication. MFA is also supported, for a more solid security consolidation of the system.

3.1.3. Shared data encryption

The data being shared is encrypted using asymmetric keys owned by the receiver's app. Each time new data is being shared from the sender to the receiver, the receiver app transparently creates a new asymmetric key pair, stores internally the private key for data decryption, and shares the public key with the sender over a QR code.

The sender encrypts the data shared using the receiver's public key. After the receiver successfully gets the encrypted data, the receiver app decrypts it automatically using the private key, saves the data locally, then immediately and automatically deletes the key pair.

The immediate key pair deletion by the receiver app is meant to eliminate the risk of a hacker stealing the private keys somehow from the receiver, and being able to decrypt a potentially leaked QR code.

3.2. Secure peer to peer connection

For large files or large data transfer, more than a few KB which could be embedded directly into a QR code, a secure peer to peer connection is established between the sender and the receiver. The sender prepares the encrypted package and hosts a temporary file sharing service through the sender app. The receiver connects to the sender's app through a secure connection, eventually solves a challenge provided by the sender (using the same key pair) to prove its identity, and then is permitted to download the data.

The secure parameters of the connection and the sender's temporary URI, necessarily for the peer to peer connection, are embedded in the QR code which the sender generates, and are encrypted using the receiver's public key, such that only the intended receiver, the owner of the matching private key, can decrypt the URI and the connection security parameters, to prevent unauthorized actors, which may have come into the possession of the sender's QR code, from connecting to the sender's app.

3.3. DevSecOps

The development of the app will use a Secure CI/ CD pipeline, to further increase the security posture of the system.

4. Future Work

This report presented just a high level design of the system architecture and the security posture of the system.

In the next iteration, for the next semester, the main focus will be on improving and consolidating the security architecture of the system, as well as developing a proof of concept for the sender and the receiver apps.

Also, the legal security concerns, considering different regulations and security frameworks, like GDPR, NIST, etc, will have a dedicated chapter in the next report, with the first purpose of identifying the security risks and documenting them.

Also, a few practical use cases for each of the 2 proposed transfer methods will be presented, with a focus on the security provided by the system, but also the weaknesses, the risks, the identified security issues, and prevention and mitigation actions.